

Security Audit Report — SentinelEdge Agent

Fecha	2026-04-01
Auditor	Claude Code (Sonnet 4.6)
Proyecto	SentinelEdge Agent
Scope	/home/ardepa/sentineledge-agent — 9 archivos Go + install.ps1 + agent.yaml
Audit anterior	2026-03-20 (0 CRITICAL · 2 HIGH · 2 MEDIUM · 2 LOW)

Executive Summary

Severidad	Cantidad
 CRITICAL	1
 HIGH	3
 MEDIUM	4
 LOW	3
Total	11

Mitigaciones confirmadas desde audit anterior

- ✔ **Command allowlist** implementado — deny-by-default con 2 capas (SHA256 + regex patterns) — `internal/allowlist/allowlist.go`
- ✔ **VaultClientSecret** removido del código fuente — se inyecta vía `SE_VAULT_CLIENT_SECRET` env var

Regresiones vs 2026-03-20

Ninguna. Los nuevos hallazgos HIGH/MEDIUM son consecuencia de la implementación del allowlist (race condition) o issues preexistentes identificados con mayor profundidad en este audit.

Compliance Scores

Framework	Score	Estado
SOC2 TSC	52.9%	Parcial
HIPAA	56.7%	Parcial
CMMC L2	41.7%	Insuficiente
ISO 27001:2022	47.9%	Parcial

CRITICAL

FINDING-01 — TLS sin certificate pinning

Severity: CRITICAL

What: Todos los clientes HTTP (`resty` en `communicator.go` , `http.Client` en `updater.go` y `vault.go`) usan la configuración TLS por defecto sin certificate pinning. Un atacante con capacidad de interceptar el tráfico (MITM en red corporativa, ARP spoofing, DNS spoofing, CA comprometida) puede suplantar `saapi.ardepa.site` .

Why it matters: El agente ejecuta comandos PowerShell en endpoints Windows de clientes MSP como SYSTEM. Vector de ataque completo: MITM → servidor falso responde a `/version` con `download_url` + `hash` controlados por el atacante → el agente descarga y ejecuta malware como SYSTEM. La SHA256 verification actual no protege porque ambos valores vienen del mismo servidor comprometido.

Fix:

1. `communicator.go` — Configurar `resty` con TLS personalizado que verifique el cert de `saapi.ardepa.site` contra un pin (hash SHA256 del cert público):

```
tlsConf := &tls.Config{
    VerifyPeerCertificate: func(rawCerts [][]byte, _ [][]*x509.Certificate)
error {
    // verificar hash del cert de saapi.ardepa.site
},
}
```

2. `updater.go` y `vault.go` — Aplicar la misma TLS config.
3. Validar que `download_url` en `/version` provenga de dominios permitidos (`github.com/ardepa710/*`) antes de hacer GET.

Controls: SOC2: CC6.7, CC6.8 | HIPAA: §164.312(e)(1), §164.312(e)(2)(i) | CMMC: SC.L2-3.13.8, SC.L2-3.13.11, IA.L2-3.5.3 | ISO 27001: A.8.20, A.8.24, A.8.5

HIGH

FINDING-02 — download_url en auto-update no validada (Binary Swap / SSRF)

Severity: HIGH

What: `updater.go` — `download(info.DownloadURL, tmpPath)` descarga desde cualquier URL que el endpoint `/version` retorne, sin validar que sea `github.com/ardepa710/*` . El hash SHA256 también viene de la misma respuesta JSON.

Why it matters: Si el servidor `/version` es comprometido o suplantado (ver FINDING-01), el atacante controla tanto `download_url` como `hash` . El agente descarga y ejecuta el binario malicioso como SYSTEM. La verificación SHA256 actual es inútil en este escenario porque el hash también viene del atacante.

Fix:

```
// updater.go — Antes de download(), validar URL:
allowed := []string{
    "https://github.com/ardepa710/sentineledge-agent/",
    "https://objects.githubusercontent.com/",
}
```

```

}
func validateDownloadURL(u string) error {
    for _, prefix := range allowed {
        if strings.HasPrefix(u, prefix) { return nil }
    }
    return fmt.Errorf("download_url not in allowlist: %s", u)
}

```

A largo plazo: firmar el binario con ed25519 y verificar la firma con clave pública embebida en el código.

Controls: SOC2: CC6.7, CC6.8, CC8.1 | HIPAA: §164.312(c)(1), §164.312(c)(2) | CMMC: SI.L2-3.14.3, CM.L2-3.4.5 | ISO 27001: A.8.8, A.8.25, A.8.28

FINDING-03 — Race condition en allowlist.approvedHashes (map sin mutex)

Severity: HIGH

What: allowlist.go — approvedHashes es un map[string]string global.

RegisterApprovedScript() escribe en el mapa. En agent.go, cada comando se ejecuta en una goroutine: go func() { a.executeCommand(cmdCopy) }(). Si múltiples comandos llegan simultáneamente y alguna ruta llama a RegisterApprovedScript() concurrentemente con Validate(), hay una race condition clásica.

Why it matters: Go detectará esto con -race y puede causar panic(concurrent map read and map write) → el servicio Windows se reinicia. O peor: corrupción silenciosa del allowlist donde comandos que deberían bloquearse pasan la validación.

Fix:

```

// allowlist.go
var (
    approvedHashes = map[string]string{}
    approvedHashesMu sync.RWMutex
)

func RegisterApprovedScript(payload, description string) {
    hash := hashString(strings.TrimSpace(payload))
    approvedHashesMu.Lock()
    defer approvedHashesMu.Unlock()
    approvedHashes[hash] = description
}

// En Validate() – reemplazar lectura del mapa:
approvedHashesMu.RLock()
desc, ok := approvedHashes[hash]
approvedHashesMu.RUnlock()

```

Controls: SOC2: CC6.1, CC7.1 | HIPAA: §164.312(c)(1) | CMMC: SI.L2-3.14.1 | ISO 27001: A.8.28, A.8.9

FINDING-04 — Vault token sin caché + error leakage en logs

Severity: HIGH

What: `vault.go` — `getToken()` se llama en cada `GetSecret()` y `StoreSecret()` sin cachear el access token. El `VaultClientSecret` nunca rota. Adicionalmente, el body completo del error de Vaultwarden se incluye en el log: `return "", fmt.Errorf("vault token error %d: %s", resp.StatusCode, string(body))`. La respuesta de error puede incluir información sensible del servidor.

Why it matters:

1. Sin rotación: si el `VaultClientSecret` se filtra, el atacante tiene acceso permanente a los tokens de todos los agentes.
2. Error leakage: el body de error va a Windows Event Logs — visible para cualquier admin local del endpoint comprometido.
3. Una autenticación OAuth por operación puede causar rate limiting en Vaultwarden bajo carga.

Fix:

```
// vault.go – Cachear el access token con expiración:
type VaultClient struct {
    // ...
    cachedToken string
    tokenExpiry time.Time
    tokenMu     sync.Mutex
}

// En getToken(): leer expires_in de la respuesta y cachear.
// En error logging: solo loguear status code, no body completo.
log.Printf("vault token error: status %d", resp.StatusCode)
```

Política operativa: rotar `VaultClientSecret` cada 90 días.

Controls: SOC2: CC6.1, CC6.3, CC7.2 | HIPAA: §164.308(a)(5)(ii)(D), §164.312(a)(2)(iii) | CMMC: IA.L2-3.5.3, AC.L2-3.1.1 | ISO 27001: A.5.17, A.8.5, A.8.15

MEDIUM

FINDING-05 — Replay attack: sin nonce ni timestamp en comandos

Severity: MEDIUM

What: `models.go` — `Command` struct no tiene timestamp de emisión ni nonce. `communicator.go` `PollCommands()` ejecuta comandos sin verificación de frescura. Comandos capturados pueden ser re-ejecutados si el token del agente es comprometido.

Why it matters: Comandos repetidos como `Restart-Service <svc>` pueden causar interrupciones en endpoints de clientes. Requiere coordinación con la API para implementar.

Fix: La API debe incluir `issued_at` (UTC timestamp) en cada `Command`. El agente debe rechazar comandos con `issued_at` anterior a `(now - 5 minutos)`. La API debe marcar comandos como ejecutados (idempotency check por ID).

Controls: SOC2: CC6.7, CC6.8 | HIPAA: §164.312(e)(2)(i) | CMMC: SC.L2-3.13.8, IA.L2-3.5.10 | ISO 27001: A.8.20, A.8.5

FINDING-06 — Temp file del update script inseguro (TOCTOU + nombre predecible)

Severity: MEDIUM

What: `updater.go` — `os.WriteFile(filepath.Join(os.TempDir(), "se_update.ps1"), ...)` usa nombre fijo predecible. Un proceso malicioso local puede sobrescribir el archivo entre escritura y ejecución (TOCTOU). El script se lanza con `-ExecutionPolicy Bypass` sin verificar integridad.

Why it matters: Proceso local con privilegios de escritura en `%TEMP%\SYSTEM` puede inyectar comandos en el script de update y ejecutarlos como SYSTEM via el agente.

Fix:

```
// updater.go - Usar nombre aleatorio:
f, err := os.CreateTemp("", "se_update_*.ps1")
if err != nil { return err }
scriptPath := f.Name()
f.Close()
```

Controls: SOC2: CC6.1, CC7.1 | HIPAA: §164.312(c)(1) | CMMC: CM.L2-3.4.1, SI.L2-3.14.1 | ISO 27001: A.8.25, A.8.28

FINDING-07 — OrgID y ColIDs de Vaultwarden hardcodedos en código fuente

Severity: MEDIUM

What: `agent.go` — Las constantes `OrgID`, `ColAgentsID`, `ColAPIID` están hardcodedas. El `VaultClientID` también está en `agent.yaml` en el repo. Estos UUIDs permiten a un atacante con acceso al repositorio conocer la estructura exacta de la organización en Vaultwarden.

Why it matters: Facilita reconocimiento y targeting. Un atacante que compromete el `VaultClientSecret` + conoce estos IDs tiene acceso directo a los ciphers de todos los agentes sin conocer la estructura.

Fix: Mover `OrgID`, `ColAgentsID`, `ColAPIID` a `agent.yaml` (o variables de entorno `SE_VAULT_ORG_ID`, etc.). Verificar que `agent.yaml` está en `.gitignore`.

Controls: SOC2: CC6.1, CC6.3 | HIPAA: §164.308(a)(4)(i) | CMMC: CM.L2-3.4.2, IA.L2-3.5.1 | ISO 27001: A.5.18, A.8.9

FINDING-08 — `install.ps1` sin parámetro obligatorio para APIKey

Severity: MEDIUM

What: `install.ps1` línea 7: `$APIKey = "your-production-api-key"`. El script no tiene mecanismo de parámetros para recibir el APIKey de forma segura. En deployments automatizados via RMM, el operador puede olvidar reemplazar el placeholder — o hardcodear la key real en el archivo y subirla al repo.

Why it matters: Deployments descuidados pueden exponer la API key en código versionado, logs de RMM, o documentación de aprovisionamiento.

Fix:

```
# install.ps1 - Añadir parámetro obligatorio:
param(
```

```
[Parameter(Mandatory=$true)]
[string]$APIKey,
[string]$TenantID = "tenant-sentineledge"
)
# Uso: .\install.ps1 -APIKey "real-key-from-vault"
```

Controls: SOC2: CC6.1, CC6.3 | HIPAA: §164.308(a)(5)(ii)(C) | CMMC: IA.L2-3.5.10, CM.L2-3.4.2 | ISO 27001: A.5.17, A.8.9

LOW

FINDING-09 — Sin límite de goroutines concurrentes para comandos

Severity: LOW

What: `agent.go` — `go func() { a.executeCommand(cmdCopy) }()` sin límite. N comandos simultáneos crean N goroutines con procesos `powershell.exe`.

Fix: Semáforo con capacidad 5:

```
const maxConcurrentCommands = 5
sem := make(chan struct{}, maxConcurrentCommands)
for _, cmd := range commands {
    cmdCopy := cmd
    sem <- struct{}{}
    go func() { defer func() { <-sem }(); a.executeCommand(cmdCopy) }()
}
```

Controls: SOC2: A1.1, CC7.1 | CMMC: SC.L2-3.13.1 | ISO 27001: A.8.16

FINDING-10 — Versión hardcodeda "0.1.0" en Register

Severity: LOW

What: `communicator.go` línea 57: `Version: "0.1.0"`. La versión nunca se actualiza tras auto-updates. El MSP no puede determinar qué versión real está corriendo, dificultando gestión de vulnerabilidades.

Fix:

```
// version.go
var Version = "dev" // sobrescrito en compilación

// Build:
// go build -ldflags "-X
github.com/sentineledge/agent/internal/version.Version=v2026.04.01-1"
```

Controls: SOC2: CC8.1 | CMMC: CM.L2-3.4.1 | ISO 27001: A.8.8

FINDING-11 — Heartbeat sin verificar respuesta — no detecta servidor falso ni soporta revocación

Severity: LOW

What: `communicator.go` — `Heartbeat()` ignora completamente la respuesta (status code y body). Impide implementar revocación de agentes vía heartbeat. Un servidor down que responde 503 pasa silenciosamente como éxito.

Fix: Verificar `resp.StatusCode()` . Definir protocolo en heartbeat response: `{"status": "ok"}` | `{"status": "revoked"}` . Si `revoked` , detener polling y loguear el evento.

Controls: SOC2: CC6.3, CC7.2 | HIPAA: §164.308(a)(5)(ii)(C) | CMMC: AC.L2-3.1.1 | ISO 27001: A.5.16, A.8.16

Remediation Priority

Prioridad	Finding	Esfuerzo	Acción
1	FINDING-03 — Race condition allowlist	~15 min	<code>sync.RWMutex</code> en <code>approvedHashes</code>
2	FINDING-06 — Temp file predecible	~5 min	<code>os.CreateTemp("", "se_update_*.ps1")</code>
3	FINDING-08 — <code>install.ps1</code> sin param	~10 min	<code>param([Mandatory]\$APIKey)</code>
4	FINDING-04 — Vault error leakage	~1h	Sanitizar log + cachear token
5	FINDING-02 — <code>download_url</code> sin validar	~30 min	Allowlist de dominios en <code>updater</code>
6	FINDING-07 — OrgIDs hardcodeados	~20 min	Mover a config + verificar <code>.gitignore</code>
7	FINDING-10 — Versión hardcodeada	~30 min	<code>ldflags</code> en build pipeline
8	FINDING-09 — Sin límite goroutines	~30 min	Semáforo en <code>tick()</code>
9	FINDING-11 — Heartbeat sin respuesta	~1h	Protocolo revocación
10	FINDING-05 — Sin nonce/timestamp	~2h	Coordinar con API
11	FINDING-01 — Sin TLS pinning	Alto	Diseño + implementación TLS config